

Report

NAVI — Comprehensive Formal Verification Report

by Asymptotic

Comprehensive Formal Verification Report

Overview

This report documents formal verification of the [NAVI Protocol](#) (X) lending contracts on [Sui](#). NAVI is a decentralized money market protocol that allows users to supply assets to earn yield, borrow against collateral, and access flash loans. It follows an index-based interest accrual design closely related to [Aave V2](#), where scaled balances track principal positions and indices accumulate interest at interaction time.

We specify and verify the lending core, the arithmetic beneath it, and the administrative surface that configures both. Beyond per-function contracts, we prove properties that hold across sequences of operations, covering how value moves between the ledger and the pool, what an operation is permitted to touch, and the exact rounding boundaries at which the intended guarantees degrade.

Table of Contents

How We Work
Report Scope
Verification Highlights
Methodology
Reproducibility
Operational and Platform Assumptions
Verification Details
Reserve Management
Operation Validation
Position Health and Liquidation Logic
Solvency and Fund Conservation
Round-Trip Exactness and Idempotency
Counterexamples and Exact Boundaries

Flash Loans

Borrow Fees

Liquidity Pools

Treasury Accounting

Incentive System

Legacy Incentive

IncentiveV2

IncentiveV3

E-mode

Protocol Gating

Lending Entry Points

Oracle Management

Oracle Price Feed Configuration

Price Registry and Updates

Oracle Provider Adaptors

Oracle Price Validation Strategy

Account Management

Version and Constants

Arithmetic Operations

Interest Rate Model and Projection

Interest Rate Kernel

Real-Time Projection

Modeling Boundaries

Conclusion and Continual Assurance

How We Work

1. Understand the protocol and identify what could go wrong.
2. Prove the invariants that matter: the ones that prevent failures, not the easy ones. We go beyond shallow verification (model-checking, easy SMT solving) and use hand-crafted Boogie/smtlib and Lean proofs for the hardest properties.
3. Publish clear reports so users can understand what is safe about a protocol.

Report Scope

We cover the NAVI lending core in full. That means the reserve state and the risk parameters written into it, the rate model and the projection layer built on top of it, and the guards standing in front of every user-facing operation. It also means the pools and treasury the ledger settles against, the incentive and borrow fee machinery, the oracle stack from configuration through runtime price validation, the efficiency mode admission logic, and the arithmetic kernel all of it rests on.

We prove that each administrative write records exactly the value it was given and disturbs nothing else, that interest and balance accounting are exact at both user and pool level, and that the caps and gates protecting the protocol hold across the writes they authorize. We also prove properties of operation sequences rather than single calls, covering the value that moves on each leg, the state each leg is permitted to reach, the exactness of a round trip and the band within which it is inexact, and the closed-form witnesses that pin where an intended invariant stops holding.

Out of scope: off-chain oracle price feed accuracy, UI and frontend, and governance process correctness beyond on-chain capability enforcement.

Verification Highlights

Reserve Parameter Isolation. Every risk parameter setter is proven to write exactly the supplied value and leave all other reserve fields unchanged. Version and pause checks guard all mutations, and the accessor surface that reads those parameters back is proven to return exactly what was written. See [Reserve Management](#).

Interest and Balance Sheet Integrity. Index updates, rate updates, and balance mutations are proven exact at both user and pool level. The `logic::update_state` function is proven end to end: supply index, borrow index, timestamp, and treasury balance all update to their expected values with compositional overflow safety. The full-reserve sweep `lending::update_state_of_all` is proven with a loop invariant composing per-asset updates. Validation guards for deposit, withdrawal, borrow, repay, and liquidation are fully specified with overflow preconditions and cap checks. See [Reserve Management](#) and [Operation Validation](#).

Fund Conservation and Frame Isolation. For deposit, withdrawal, borrow, and repay we prove that the change in the user's scaled entry, the change in the reserve total, and the coin actually moved into or out of the pool are the same quantity. We also prove the frame side of the same question: each operation writes only the reserve and the user it names, leaving every other reserve, every other user, and the opposite side of the ledger untouched. The repay path is proven to conserve the caller's coin exactly across the split, the pool netting, and the refund. See [Solvency and Fund Conservation](#).

Round-Trip Exactness with a Bounded Band. A full repay is proven to clear a debt to exactly zero rather than to a residual dust value, and the borrow-then-repay round trip at a single tick is proven to return the position to its entry state. Where a round trip is not exact, the loss is bounded rather than assumed away: deposit followed by withdrawal is proven to under-pay the user by at most one index unit and never to over-pay. See [Round-Trip Exactness and Idempotency](#).

Refuted Invariants. Two defects are proven to break intended protocol invariants, in each case with a closed-form witness rather than an inferred failure. The borrow guard and the borrow write use mismatched rounding, so a borrow can book more scaled debt than the reserve's scaled supply once the borrow index has grown. The withdrawal dust

sweep credits the treasury without debiting anyone, so the same value is owed to the depositor and to the treasury at once. See [Counterexamples and Exact Boundaries](#).

Borrow Fee Precedence. The borrow fee rate resolves through a user override, then an asset rate, then the global default. We prove that the resolution respects that order at each level, that removing a rate falls back to the next one, and that the empty-table starting state resolves to the configured default. See [Borrow Fees](#).

Protocol Gating. Pause flags, version gates, and the E-mode active flag are proven to be last-write-wins toggles that are idempotent under repetition. The gated operations are proven to abort when the gate is closed, on the storage side, the oracle side, and the E-mode admission path. See [Protocol Gating](#) and [E-mode](#).

Interest Rate Model and Projection. See [Interest Rate Model and Projection](#).

Incentive Coverage. Reward fund accounting for both IncentiveV2 and V3 is proven exact, along with pool management, querying, and freezing under loop invariants. Reward settlement and the no-double-claim property are proven at the rule level, and the reward fund round trip is proven to conserve funds. See [Incentive System](#).

Oracle Integrity. The oracle management surface is proven in full, together with the price feed configuration and lookup layer, the Pyth, Supra, and Switchboard adaptors, and the decimal scaling that connects them. Price freshness, price difference classification, range and history checks, and the ordering of the configured bounds across a chain of admin writes are all proven. See [Oracle Management](#).

Arithmetic Safety. All ray-math and safe-math operations are proven overflow-safe with exact computation postconditions. Compounded interest is proven with compositional pure helpers for each Taylor term, the ray algebra laws are proven directly, and the wad-to-ray conversion is proven to round trip exactly. See [Arithmetic Operations](#).

Methodology

Proof structure. Each specification mirrors the signature of its target function. The spec body establishes preconditions (`asserts -- verified`

by the prover, `requires` -- assumed, typically for overflow bounds that are structurally guaranteed but expensive to re-derive), invokes the target, then states postconditions (`ensures`) that the prover must discharge.

Abort modeling. Where feasible, specifications fully model abort conditions using `asserts` preconditions that the prover verifies, proving both that the function succeeds when preconditions hold and aborts otherwise. Some legacy or deprecated functions are proven to always abort, confirming they cannot be called successfully.

Abort handling. For composed user-facing operations where exhaustive abort modeling is impractical, specifications verify properties of successful executions only. Abort conditions are still enforced by implementation-level assertions that revert the transaction. The spec proves correct state updates on success.

Access control. Privileged operations take capability objects (e.g. `StorageAdminCap` , `OwnerCap` , `OracleAdminCap`) as parameters. In `Move`, function signatures are enforced at the bytecode level: a caller cannot invoke a privileged function without possessing the required capability. This is a language-level guarantee, not a property the prover re-derives.

Scenario Specifications. Some properties are statements about a sequence of operations rather than about one function, and no single implementation function exists to attach them to. These are stated as scenario specifications that target no client function and instead replay the operation sequence against real protocol state, calling the real mutators under their already proven contracts. This is the technique behind the conservation, frame, round-trip, and gating results.

Invariant Catalogue. The properties proven here are drawn from a catalogue of protocol invariants, each carrying a formal statement and the call sites at which it must be established or preserved. Where an invariant cannot be discharged, we record a closed-form witness for the failure rather than leaving the entry silent, so a refuted property is a proven statement in its own right.

Known Limitations. A number of functions are modeled at the structural level rather than with exhaustive postconditions, and such cases are tracked in the verification dashboard.

Reproducibility

Protocol revision: 96fa2242e9c909fb8ffffc29ca684c296d1562bb

Sui Prover version: 1.5.3

Operational and Platform Assumptions

These proofs rely on the correctness of the Sui execution environment: the Move VM, bytecode verifier, and runtime resource/capability enforcement are assumed to behave according to their specifications. Standard network conditions for protocol operation are assumed, including validator correctness, epoch progression, and chain liveness.

The trust model assumes that external data and governance controls are sound in production. Oracle prices feeding into health checks and liquidation computations are assumed timely and accurate. Privileged administrative capabilities are assumed held and operated by intended governance actors, including the capabilities that designate protected liquidation users and designated liquidators.

Several risk parameter orderings are maintained by the administrator rather than enforced on chain. Loan-to-value is intended to stay at or below the liquidation threshold, and the configured price bounds are intended to stay correctly ordered, but the individual setters carry no cross-parameter guard. We prove that the intended orderings survive the setter chains that maintain them, under the assumption that each write respects the ordering.

The guarantees here cover modeled contract behavior, including explicitly modeled abort paths and proven postconditions. They do not cover off-chain infrastructure, operational processes, or source-level properties not expressed in the specifications.

Verification Details

Reserve Management

The `storage` module maintains per-reserve state: indices, balances, rates, risk parameters, and user accounting. All mutations are version-

gated and reserve-existence-checked. We prove the full set of accessor and mutation functions.

Version and pause guards. We prove that `when_not_paused` aborts when the storage is paused, and `version_verification` aborts when the stored version does not match `constants::version()` .
`version_migrate` requires the current version to be strictly less than the target and sets it to `constants::version()` .

Specifications: Version and Pause Guards

<code>storage_specs::when_not_paused</code>

<code>storage_specs::version_verification</code>
--

<code>storage_specs::set_pause</code>

<code>storage_specs::version_migrate</code>

Reserve parameter updates. Each reserve parameter setter is proven to write exactly the supplied value with isolation: unrelated fields in the same reserve are unaffected. Parameters bounded by RAY assert that bound in their preconditions.

Specifications: Reserve Parameter Updates

<code>storage_specs::set_supply_cap</code>
--

<code>storage_specs::set_borrow_cap</code>
--

<code>storage_specs::set_ltv</code>

<code>storage_specs::set_treasury_factor</code>

<code>storage_specs::set_base_rate</code>

<code>storage_specs::set_multiplier</code>
--

<code>storage_specs::set_jump_rate_multiplier</code>
--

<code>storage_specs::set_reserve_factor</code>
--

<code>storage_specs::set_optimal_utilization</code>

<code>storage_specs::set_liquidation_ratio</code>

<code>storage_specs::set_liquidation_bonus</code>

<code>storage_specs::set_liquidation_threshold</code>

Interest rate and index accounting. `update_state` proves both indices update to the supplied values, the last-update timestamp is recorded, and the treasury balance increases by exactly `scaled_treasury_amount` (with an overflow precondition). Both require the version check and reserve existence. `increase_balance_for_pool` proves pool-level supply and borrow totals each increase by exactly the supplied amounts.

Specifications: Interest Rate and Index Accounting

<code>storage_specs::update_interest_rate</code>
--

<code>storage_specs::update_state</code>
--

<code>storage_specs::increase_balance_for_pool</code>

User balance accounting. `increase_supply_balance` and `decrease_supply_balance` prove that both per-user supply balance and the pool-level total supply change by exactly the supplied amount (with matching overflow and underflow preconditions).

`increase_borrow_balance` and `decrease_borrow_balance` prove the analogous properties for borrow positions. `increase_treasury_balance` proves treasury balance increases by exactly the supplied amount.

Specifications: User Balance Accounting

<code>storage_specs::increase_supply_balance</code>

<code>storage_specs::decrease_supply_balance</code>

<code>storage_specs::increase_borrow_balance</code>

<code>storage_specs::decrease_borrow_balance</code>

<code>storage_specs::increase_treasury_balance</code>

<code>storage_specs::increase_total_supply_balance</code>

<code>storage_specs::decrease_total_supply_balance</code>

User asset tracking. `update_user_loans` and `remove_user_loans` prove correct table state after insertion and removal of borrow positions.

`update_user_collaterals` and `remove_user_collaterals` prove the same for collateral positions.

Specifications: User Asset Tracking

<code>storage_specs::update_user_loans</code>

<code>storage_specs::remove_user_loans</code>

<code>storage_specs::update_user_collaterals</code>

<code>storage_specs::remove_user_collaterals</code>

Reserve initialization. `init_reserve` has a specification with `ignore_abort` covering dynamic object creation.

`init_for_main_market` and `init_borrow_weight_for_main_market` prove correct initialization. `create_new_market` is verified end to end.

`reserve_validation` is proven with loop invariants (via `requires`) asserting that the reserves table contains all registered indices and the supplied coin type is not already registered.

Specifications: Reserve Initialization

<code>storage_specs::init_reserve</code>
--

<code>storage_specs::init_for_main_market</code>
--

<code>storage_specs::init_borrow_weight_for_main_market</code>
--

<code>storage_specs::create_new_market</code>

<code>storage_specs::reserve_validation</code>
--

Liquidation guard. `is_liquidatable` and `when_liquidatable` prove the liquidation eligibility predicate and its abort guard. The liquidation protection configuration, covering both the designated liquidators and the protected users, is proven to initialize and mutate correctly.

Specifications: Liquidation Guard

<code>storage_specs::is_liquidatable</code>

<code>storage_specs::when_liquidatable</code>

<code>storage_specs::init_protected_liquidation_fields</code>

<code>storage_specs::set_designated_liquidators</code>
--

<code>storage_specs::set_protected_liquidation_users</code>

Borrow weight. `set_borrow_weight` and `remove_borrow_weight` prove that the borrow weight for a given asset is set or removed from the storage table.

Specifications: Borrow Weight

<code>storage_specs::set_borrow_weight</code>

<code>storage_specs::remove_borrow_weight</code>
--

<code>manage_specs::set_borrow_weight</code>
--

<code>manage_specs::remove_borrow_weight</code>

Risk parameter ordering. Loan-to-value must not exceed the liquidation threshold, otherwise a position could be opened already eligible for liquidation. The individual setters carry no cross-parameter guard, so the ordering is an administrative discipline rather than an on-chain check. We prove that the discipline is sufficient: given writes that respect the ordering, it survives the setter chain at the reserve level and at the E-mode level, and each setter is proven not to disturb the other parameter.

Specifications: Risk Parameter Ordering

<code>ltv_lt_ordering_scenario_specs::reserve_ltv_lt_ordering_scenario</code>

<code>ltv_lt_ordering_scenario_specs::emode_ltv_lt_ordering_scenario</code>

Reserve state accessors. The reserve read surface is proven alongside the mutators, so that a value written by a setter is proven to be the value a caller reads back. This matters because the health, rate, and validation layers all consume reserve state exclusively through these accessors. The specifications below are representative, and the full accessor surface is available in the verification dashboard.

Specifications: Reserve State Accessors (Representative)

<code>storage_specs::get_index</code>
<code>storage_specs::get_total_supply</code>
<code>storage_specs::get_user_balance</code>
<code>storage_specs::get_borrow_rate_factors</code>
<code>storage_specs::get_liquidation_factors</code>
<code>storage_specs::get_asset_ltv</code>
<code>storage_specs::get_supply_cap_ceiling</code>
<code>storage_specs::get_borrow_cap_ceiling_ratio</code>
<code>storage_specs::get_current_rate</code>
<code>storage_specs::get_last_update_timestamp</code>
<code>storage_specs::get_reserves_count</code>
<code>storage_specs::get_user_assets</code>

Market and ownership. `get_market_id` and `get_storage_market_info` prove that the market identity a caller reads matches the one recorded at initialization, which is the fact the cross-object consistency checks depend on. `mint_owner_cap` proves the administrative capability is minted and transferred, and `destory_user` proves the user record is removed.

Specifications: Market Identity and Ownership

<code>storage_specs::get_market_id</code>
<code>storage_specs::get_storage_market_info</code>
<code>storage_specs::get_oracle_id</code>
<code>storage_specs::get_coin_type</code>
<code>storage_specs::mint_owner_cap</code>
<code>storage_specs::destory_user</code>

Operation Validation

The `validation` module guards every user-facing operation with arithmetic and protocol-level checks. We prove all five validators fully.

Deposit validation. We prove that `validate_deposit` aborts unless: the reserve exists, the coin type matches the registered type, the amount is nonzero, the scaled supply balance is within `u256` bounds, the deposit does not overflow the scaled total, and the post-deposit supply does not exceed `supply_cap_ceiling`. All precondition arithmetic is overflow-checked.

Withdrawal validation. We prove that `validate_withdraw` aborts unless: the reserve exists, the coin type matches, the amount is nonzero, the scaled supply balance is within bounds, the withdrawal does not underflow below the scaled borrow balance (preserving solvency), and the withdrawn amount does not exceed available liquidity.

Borrow validation. We prove that `validate_borrow` aborts unless: the reserve exists, coin type matches, amount is nonzero, the borrow cap ceiling ratio is not exceeded, and all arithmetic stays within bounds.

Repay and liquidation validation. `validate_repay` and `validate_liquidate` are proven with corresponding precondition sets checking reserve existence, type matching, and arithmetic safety.

Specifications: Operation Validation

```
validation_specs::validate_deposit
```

```
validation_specs::validate_withdraw
```

```
validation_specs::validate_borrow
```

```
validation_specs::validate_repay
```

```
validation_specs::validate_liquidate
```

Caps survive the write they guard. A cap check is only meaningful if it still holds after the operation it authorized has run. The guards are checked against a raw amount while the write records a scaled one, so the two are related through a rounding step and the cap could in principle be exceeded by the write it permitted. We prove that it is not: replaying the real ledger write against the state the guard just established, the supply cap invariant still holds afterwards, and the supply index and the ceiling itself are proven untouched across the write so the invariant is read against the same values the guard was checked against. The borrow side replays both of the borrow guards, the ratio cap and the strict solvency ordering, against the same write.

Specifications: Cap Enforcement Across the Write

```
cap_scenario_specs::deposit_cap_scenario
```

Position Health and Liquidation Logic

The `logic` module computes portfolio health using oracle prices and reserve risk parameters. Health and valuation functions are specified with `ignore_abort` due to cross-module oracle calls.

Health and health factor. `is_health` is proven (modulo abort) to return the correct health boolean, and each health factor entry point is proven end to end for successful executions. `calculate_avg_ltv` computes the collateral-value-weighted average loan-to-value across collateral types. The membership predicates that decide which assets enter that aggregate are proven alongside the per-asset balances it is built from.

Collateral and loan valuation. The aggregate USD value of a user's collateral and the aggregate value of their debt are each proven to be computed correctly. The health-weighted variants and the borrow-weight-adjusted variant are proven the same way, so the quantities the liquidation decision reads are the quantities the risk parameters define.

Interest state update. `update_state` proves the full interest accrual pipeline: given the elapsed time since the last update, the supply index updates via linear interest, the borrow index updates via third-order Taylor-approximated compounded interest, the last-update timestamp is set to the current millisecond clock reading, and the treasury balance increases by the scaled treasury amount. The two time quantities in that list are not the same quantity: the elapsed time driving both indices is the millisecond difference floored to whole seconds, while the stored timestamp is the unfloored reading. The specification pins both, so the pairing is on the record rather than smoothed over, and its consequences are treated in the accompanying security review. The proof composes the already proven contracts of `calculate_compounded_interest`, `calculate_linear_interest`, `ray_mul`, `ray_div`, and `increase_total_supply_balance` through `uninterpreted` helpers for the compounded interest terms, so no step re-derives the arithmetic beneath it.

Specifications: Position Health and Liquidation Logic

`logic_specs::is_health`

`logic_specs::user_health_factor`

`logic_specs::user_health_factor_batch`

`logic_specs::dynamic_liquidation_threshold`

<code>logic_specs::user_health_collateral_value</code>
<code>logic_specs::user_health_loan_value</code>
<code>logic_specs::weighted_user_health_loan_value</code>
<code>logic_specs::user_loan_value</code>
<code>logic_specs::user_collateral_value</code>
<code>logic_specs::calculate_avg_ltv</code>
<code>logic_specs::user_collateral_balance</code>
<code>logic_specs::user_loan_balance</code>
<code>logic_specs::is_collateral</code>
<code>logic_specs::is_loan</code>
<code>logic_specs::cumulate_to_supply_index</code>
<code>logic_specs::update_interest_rate_consistency</code>

Borrow power. The borrow guard is expressed in terms of an average loan-to-value and a health factor rather than as a direct solvency comparison, so it is not obvious from the code that passing it implies the position is solvent after the borrow. We prove that reduction: pinned to the proven functional models of the three quantities the guard reads, the guard is equivalent to strict post-borrow solvency. The reduction consumes the ordering between average loan-to-value and average liquidation threshold, which holds per asset and lifts to the aggregate because both averages use the same weights over the same asset vector.

Liquidation clamp and bonus split. The liquidation payout is clamped before it is split, and the split multiplies by an effective bonus. We prove the clamp chain and the split exactly as the implementation writes them, against real ray arithmetic and with the two subtractions in the body discharged rather than waived, so the clamp is proven to be what makes them safe. Separately we prove the bonus is bounded on both branches of the E-mode override:

$$\text{effective_bonus} \leq \text{RAY}$$

This bound is what allows the split to be stated without an assumption. It is discharged from the reserve parameter invariants on the reserve branch and from the E-mode pair invariants on the E-mode branch, both of which are established and preserved by the setters proven in [Reserve Management](#) and [E-mode](#).

Specifications: Borrow Power and Liquidation Split

<code>logic_scenario_specs::borrow_power_health_scenario</code>
<code>logic_scenario_specs::liquidation_clamp_split_scenario</code>
<code>logic_scenario_specs::liquidation_effective_bonus_bounded_scenario</code>

Solvency and Fund Conservation

A money market is safe only if every state-changing operation moves value conservatively. What leaves one ledger must arrive in another, no path may mint or destroy a claim, and an operation must not reach state it was never asked to touch. These are statements about a whole operation rather than about any one function, so we prove them by replaying each operation's write set against real protocol state, calling the real storage mutators under their already proven contracts.

Per-operation ledger deltas. For deposit, withdrawal, borrow, and repay, the operation reads the reserve index, scales the raw amount by it, and writes the scaled result to two places at once: the user's entry and the reserve total. We prove that both move by the same quantity, and that the quantity is exactly the scaled amount rather than an approximation of it.

$$\Delta_{\text{user_scaled}} = \Delta_{\text{total_scaled}} = \text{ray_div}(\text{amount}, \text{index})$$

The divisor is the reserve index, which the protocol maintains at or above `RAY` and therefore never at zero, so the scaling is always well defined. `delta_refines_ledger_step_scenario_spec` proves the per-operation delta refines the abstract ledger step, which is what allows the four legs to be reasoned about as instances of one rule.

Specifications: Per-Operation Ledger Deltas

<code>mi10_delta_scenario_specs::deposit_delta_scenario</code>
<code>mi10_delta_scenario_specs::withdraw_delta_scenario</code>
<code>mi10_delta_scenario_specs::borrow_delta_scenario</code>
<code>mi10_delta_scenario_specs::repay_delta_scenario</code>
<code>mi10_delta_scenario_specs::delta_refines_ledger_step_scenario</code>

Cash link. The delta proofs establish that the ledger is internally consistent. They do not establish that real coin moved. The cash link proofs close that gap on all four legs by replaying the money path of the corresponding entry point: on a deposit the pool balance rises by exactly what the ledger recorded, on a borrow it falls by exactly what the debt rose by, on a repay it rises by exactly the debt cancelled, and on a withdrawal it falls by exactly what the supplier gave up.

Specifications: Cash Link Between Ledger and Pool

cash_link_scenario_specs::deposit_cash_link_scenario
cash_link_scenario_specs::withdraw_cash_link_scenario
cash_link_scenario_specs::borrow_cash_link_scenario
cash_link_scenario_specs::repay_cash_link_scenario

Repay conservation. Repayment is the one path where the caller can hand in more than is owed, so it carries a clamp, a split, and a refund. We prove the refund is exactly the overshoot, that the pool keeps exactly the debt it cancelled and nets against the split amount rather than against the caller's whole coin, and that a full repay takes the asset off the user's loan list. Taken together the four legs account for every unit the caller supplied:

$$\text{pool_kept} + \text{refund} + \text{remainder} = \text{paid_in}$$

No unit of the caller's coin is created or destroyed on the repay path.

Specifications: Repay Conservation

mi11_repay_scenario_specs::repay_refund_scenario
mi11_repay_scenario_specs::repay_split_conservation_scenario
mi11_repay_scenario_specs::repay_pool_net_scenario
mi11_repay_scenario_specs::repay_clears_loan_flag_scenario

Frame isolation. Conservation says the value that moved is accounted for. Framing says nothing else moved at all. For each of the five core operations we replay the complete ledger write set and prove that every other reserve, every other user, and the opposite side of the ledger are left exactly as they were. The liquidation case is the strongest of the five because it legitimately touches two assets and two parties, and the frame pins precisely which four entries may change.

Specifications: Frame Isolation

mi12_frame_scenario_specs::deposit_frame_scenario
mi12_frame_scenario_specs::withdraw_frame_scenario
mi12_frame_scenario_specs::borrow_frame_scenario
mi12_frame_scenario_specs::repay_frame_scenario
mi12_frame_scenario_specs::liquidate_frame_scenario

Round-Trip Exactness and Idempotency

Round-trip proofs license claims about sequences that per-function contracts cannot reach. Without them we could say a deposit increases a balance by the deposited amount. With them we can say what a user gets back.

Debt clears exactly. A full repay writes the scaled debt back through a divide and a multiply that both round half up, so the natural expectation is a residual of one unit rather than zero. We prove that expectation wrong in the favorable direction: the full repay branch clears the debt to exactly zero for any live position, and the borrow-then-repay round trip at a single tick returns the ledger and the user's loan listing to their entry state. There is no dust position left behind to block a later operation.

Deposit and withdrawal are banded, never favorable to the user. The supply side does not clear exactly, and we state the boundary rather than assume it away. Depositing an amount and reading it back loses at most one index unit, and withdrawing in the same tick is proven never to over-pay and to under-pay by at most the same band:

$$0 \leq \text{amount} - \text{withdrawn} \leq \frac{\text{supply_index}}{\text{RAY}} + 1$$

RAY is a nonzero constant, so the bound is always defined, and because the protocol maintains the supply index at or above RAY the shortfall is a single token unit while the index is below twice RAY, widening only as interest accrues. The asymmetry is the point: rounding can never manufacture funds for the user, and the residual always favors the protocol.

Accrual is idempotent at a fixed clock reading. Applying interest accrual twice against the same Clock value is proven to leave both indices, the stored last-update timestamp, the reserve total supply and the treasury balance as the first application left them. The hypothesis is that elapsed time, floored to whole seconds, is zero, so the second application computes unit interest factors and writes back the values it read. This is what licenses treating a single tick as a point in the sequence proofs above, where a borrow leg and a repay leg both read the same index, with a tick meaning one clock reading rather than one second. The scope is worth stating plainly: this is a claim about repetition at one reading, not a claim that two accruals less than a second apart are free of loss. The elapsed time driving both indices is floored to whole seconds while the stored timestamp is written from the unfloored millisecond clock, so

accruals at two different readings inside the same second advance the stored timestamp with no interest accrued and no carry forward of the remainder. The specifications state that pairing rather than assuming it away, and its consequences are treated in the accompanying security review.

Specifications: Ledger Round Trips

<code>mi30_roundtrip_scenario_specs::full_repay_clears_debt_scenario</code>
<code>mi30_roundtrip_scenario_specs::borrow_repay_clears_roundtrip_scenario</code>
<code>logic_scenario_specs::deposit_withdraw_roundtrip_scenario</code>
<code>logic_scenario_specs::deposit_withdraw_residual_scenario</code>
<code>logic_scenario_specs::borrow_repay_roundtrip_scenario</code>
<code>logic_scenario_specs::update_state_idempotency_scenario</code>

Balance and membership algebra. At the storage layer we prove the composition laws the higher-level round trips rest on. Two increases followed by one matching decrease restore both the user entry and the reserve total to their entry values, supply operations never touch the borrow side, and operations on one user never touch another. The asset listing behaves the same way: adding an asset yields membership, adding it again is a no-op, and removing it restores the original vector exactly.

Specifications: Balance and Membership Algebra

<code>storage_scenario_specs::balance_roundtrip_scenario</code>
<code>storage_scenario_specs::membership_scenario</code>

Counterexamples and Exact Boundaries

Two defects refute four entries in the catalogue. The borrow step breaks MI-01 (reserve solvency), and with it the weaker non-strict form of the same property carried as MI-24. The withdrawal dust sweep breaks MI-09 (treasury minted consistently and always redeemable) and, in the same write, MI-02 (TokenBalance conservation). In each case we prove a closed-form witness with concrete in-domain values rather than reporting a failed proof attempt, and we state the exact boundary at which the property stops holding rather than flagging the write and stopping there.

The borrow step does not re-establish solvency (MI-01). The borrow guard and the borrow write use different arithmetic. The guard adds a

raw amount to a scaled balance, while the write adds a scaled amount to a raw one, and both the scaling and the unscaling round half up. The strict inequality in the guard buys exactly one unit of headroom, and the round trip through the two roundings can consume more than that once the borrow index has grown past roughly twice RAY .

```
guard  ray_mul(tb,bi) + amount < ray_mul(ts,si)
write  tb' = tb + ray_div(amount,bi)
goal   ray_mul(tb',bi) < ray_mul(ts,si)
```

We prove a witness at which the guard passes and the goal is strictly false. Both indices sit at or above RAY , so the witness is a state the accrual path can actually produce rather than an out-of-domain artifact. The same witness refutes the weaker non-strict form of the invariant, MI-24, which matters because that weaker form is threaded as a hypothesis through the utilization and borrow rate specifications.

The withdrawal dust sweep breaks redeemability (MI-09). When a withdrawal leaves a small remainder, the implementation credits that remainder to the treasury. The withdrawal itself is conservative and debits both the user entry and the reserve total, but the dust credit raises the treasury balance with no matching debit anywhere. The leftover is then owed twice, once to the depositor whose entry still holds it and once to the treasury, which an administrator can drain. That second reading of the same value is the conservation break recorded as MI-02.

We prove the boundary as a biconditional, so the invariant survives the sweep exactly when the treasury credit does not exceed the depositor's remaining claim. We prove separately that the credit always does exceed it once interest has accrued, because the credit is denominated in actual units while the remaining claim is denominated in scaled ones. Together the two results are a reachability proof for a treasury balance larger than the reserve total it lives inside.

Specifications: Counterexamples and Exact Boundaries, MI-01, MI-02, MI-09 and MI-24

```
mi01_solvency_step_scenario_specs::mi01_borrow_step_counterexample_scenario
```

```
mi09_treasury_scenario_specs::withdraw_dust_treasury_break_scenario
```

```
mi09_treasury_scenario_specs::dust_sweep_unit_mismatch_scenario
storage_scenario_specs::withdraw_dust_conservation_break_scenario
```

Flash Loans

The flash loan module provides per-asset flash loan configuration with fee splits between suppliers and treasury.

Version lifecycle. `version_verification` is proven to abort unless the config version equals `constants::version()`. `version_migrate` is proven to set the version to `constants::version()` when the current version is strictly lower. `init_config_for_main_market` proves the market ID dynamic field is created exactly once (asserts it does not already exist, proves it exists after).

Configuration and rate management. Each flash loan asset setter, covering the two fee rates and the loan size bounds, is proven to write exactly the supplied value to the asset config.

Specifications: Flash Loans

`flash_loan_specs::version_verification`

`flash_loan_specs::version_migrate`

`flash_loan_specs::init_config_for_main_market`

`flash_loan_specs::create_config`

`flash_loan_specs::get_asset`

`flash_loan_specs::get_market_id`

`flash_loan_specs::parsed_receipt`

`manage_specs::set_flash_loan_asset_rate_to_supplier`

`manage_specs::set_flash_loan_asset_rate_to_treasury`

`manage_specs::set_flash_loan_asset_min`

`manage_specs::set_flash_loan_asset_max`

`manage_specs::create_flash_loan_config`

`manage_specs::create_flash_loan_config_with_storage`

Loan and repayment conservation. A flash loan is only safe if the pool cannot end the transaction with less cash than it began with. We prove the round trip end to end, taking the loan through the `real` function and modeling the repayment by the primitives the implementation calls, in the implementation's order and with the implementation's arguments:

$$\begin{aligned}\text{pool_post} &= \text{pool_pre} + \text{fee_to_supplier} \\ \text{treasury_post} &= \text{treasury_pre} + \text{fee_to_treasury}\end{aligned}$$

The pool strictly gains the supplier fee across the round trip, and the treasury gains exactly the treasury fee and nothing more. The receipt returned by the loan carries no ability to be stored, copied, or dropped,

so the only way to discharge it on chain is the repayment function, which re-checks the borrower and the pool it was issued against.

`parsed_receipt` proves the receipt fields a caller reads are the ones the loan recorded, and `flash_config_establish_mi14_scenario_spec` proves the fee configuration predicate the conservation argument rests on is established when an asset is registered.

Specifications: Flash Loan Conservation

```
flash_loan_scenario_specs::flash_loan_roundtrip_scenario
```

```
flash_loan_scenario_specs::flash_config_establish_mi14_scenario
```

Borrow Fees

Borrowing carries a fee that settles into a balance held separately from reserve liquidity. Three rates can apply to a given borrow: a per-user override, a per-asset rate, and a global default. Which one is charged is decided by a lookup chain rather than by an explicit branch, so the precedence is a property of the tables rather than of the code.

Precedence. We prove the chain resolves in the intended order. After setting only the global rate, the fee uses it. After setting the asset rate, the asset rate wins. After removing the asset rate, the fee falls back to the global rate. The user-level scenario proves the same relationship one level higher, with the user override taking precedence over both. The charged fee is then

$$\text{fee} = \frac{\text{amount} \times \text{fee_rate}}{\text{percentage_benchmark}}$$

where `percentage_benchmark` is the fixed constant ten thousand, proven by `constants_specs::percentage_benchmark_spec`, so the denominator is never zero.

Specifications: Borrow Fee Precedence

```
fee_precedence_scenario_specs::fee_precedence_asset_scenario
```

```
fee_precedence_scenario_specs::fee_precedence_user_scenario
```

```
incentive_v3_specs::borrow_fee_empty_tables_establish_mi19_scenario
```

Rate configuration and collection. Each rate setter and remover is proven to write or clear exactly the entry it names, at both the administrative layer and the incentive layer. `withdraw_borrow_fee` is proven to require a `BorrowFeeCap` and to transfer the requested amount from the fee balance to the recipient. `init_borrow_fee_fields` proves

the fee tables are created on the incentive object, and `get_borrow_fee_v2` proves the rate a borrow actually reads.

Specifications: Borrow Fee Configuration

`manage_specs::withdraw_borrow_fee`

`manage_specs::mint_borrow_fee_cap`

`manage_specs::set_borrow_fee_rate`

`manage_specs::set_asset_borrow_fee_rate`

`manage_specs::set_user_borrow_fee_rate`

`manage_specs::remove_incentive_v3_asset_borrow_fee_rate`

`manage_specs::remove_incentive_v3_user_borrow_fee_rate`

`incentive_v3_specs::set_asset_borrow_fee_rate`

`incentive_v3_specs::set_user_borrow_fee_rate`

`incentive_v3_specs::remove_asset_borrow_fee_rate`

`incentive_v3_specs::remove_user_borrow_fee_rate`

`incentive_v3_specs::init_borrow_fee_fields`

`incentive_v3_specs::get_borrow_fee_v2`

`incentive_v3_specs::withdraw_borrow_fee`

Liquidity Pools

The `pool` module holds per-asset balances (principal and treasury) and manages the link to the Sui liquid staking pool for SUI pools.

Balance operations. `withdraw_balance` proves the returned balance equals the requested amount and `pool::get_balance` decreases by that amount. The preconditions assert the balance is sufficient, the pool is not managed, and the market ID exists. `withdraw_treasury` proves the treasury balance decreases by the amount and that the transfer is issued to the correct recipient address.

Pool manager configuration. `init_sui_pool_manager` proves the SUI pool manager is initialized.

Decimal and amount conversion. The protocol holds every asset in a normalized nine-decimal scale internally while pools hold coins in their native decimals, so every deposit and withdrawal crosses a scaling boundary. `convert_amount` is specified for the underlying fixed-point scaling, and `normal_amount` and `unnormal_amount` are each proven against their exact formula with the scale-up overflow guard discharged rather than waived. We then prove the pair is an exact bijection for any pool of nine decimals or fewer:

`unnormal_amount(normal_amount(amount)) = amount`

Nothing is lost crossing into the protocol's internal scale and back out again, so the normalization layer cannot be a source of the rounding drift the ledger proofs bound elsewhere.

Liquid staking integration. The SUI pool's interactions with the Volo liquid staking layer are specified modulo abort, covering staking, unstaking, validator weighting, and the treasury path that holds the staked position.

Specifications: Liquidity Pools

`pool_specs::create_pool`

`pool_specs::init_pool_for_main_market`

`pool_specs::deposit_balance`

`pool_specs::withdraw_balance`

`pool_specs::withdraw`

`pool_specs::withdraw_treasury`

`pool_specs::get_market_id`

`pool_specs::get_coin_decimal`

`pool_specs::uid`

`pool_specs::init_sui_pool_manager`

`pool_specs::enable_manage`

`pool_specs::disable_manage`

`pool_specs::refresh_stake`

`pool_specs::withdraw_vsui_from_treasury`

`pool_specs::set_target_sui_amount`

`pool_specs::unstake_vsui`

`pool_specs::set_validator_weights_vsui`

`pool_specs::direct_deposit_sui`

Specifications: Normalization Round Trip

`pool_specs::convert_amount`

`pool_specs::normal_amount`

`pool_specs::unnormal_amount`

`pool_specs::normalization_roundtrip_scenario`

Fund round trip. Depositing a coin into a pool is proven to add exactly its value to the pool balance, and withdrawing the same amount immediately afterwards is proven to restore the original balance. Combined with the

cash link proofs in [Solvency and Fund Conservation](#), this covers both directions of the pool's relationship with the ledger.

Specifications: Pool Fund Round Trip

```
funds_roundtrip_scenario_specs::pool_roundtrip_scenario
```

Treasury Accounting

The treasury holds the protocol's share of accrued interest. Its balance is denominated in the same scaled units as the reserve total it lives inside, and the safety property is redeemability: the treasury's claim must never exceed the total it is drawn from, otherwise an administrator could withdraw value that belongs to depositors.

The accrual mint preserves redeemability. Interest accrual closes with two storage writes, an index update carrying the scaled treasury amount and a matching increase of the reserve total. These are written far apart in different styles and nothing in the code marks them as a pair, yet the pairing is exactly what keeps the treasury inside the total. We prove that both sides move by the same amount, so the treasury's claim never outgrows the total that backs it.

The drain preserves it too. Withdrawal from the treasury is bounded on both sides and removes the same scaled amount from the treasury and from the reserve total. We prove that given exactly the contract the withdrawal publishes, redeemability holds at the post state, so the drain cannot strand the fee that remains. This is not bookkeeping: the second leg on its own breaks redeemability for any drain larger than the entering slack, and it is the pairing that makes the operation safe.

Specifications: Treasury Accounting

```
mi09_treasury_scenario_specs::treasury_mint_preserves_scenario
```

```
mi09_treasury_scenario_specs::treasury_drain_step_scenario
```

```
storage_scenario_specs::accrual_mint_ledger_scenario
```

```
storage_specs::increase_treasury_balance
```

```
storage_specs::withdraw_treasury
```

```
storage_specs::withdraw_treasury_v2
```

```
storage_specs::get_treasury_balance
```

```
storage_specs::get_treasury_factor
```

```
pool_specs::deposit_treasury
```

```
pool_specs::get_treasury_sui_amount
```

The one path that does not preserve redeemability is the withdrawal dust sweep, which is stated with its exact boundary in [Counterexamples and Exact Boundaries](#).

Incentive System

NAVI supports a layered incentive system. IncentiveV2 provides pool-based emission programs, and IncentiveV3 supersedes it with per-rule reward configuration and a borrow-fee mechanism.

Legacy Incentive

The legacy incentive module (`incentive`) is deprecated.

`get_pool_info` proves correct extraction of pool start time, end time, rate, and oracle ID from indexed pool vectors, with preconditions asserting table existence and vector length consistency.

`get_pool_count` proves the count matches the registered pools, and `earned` proves the accrued reward a user can read. Ownership and administration transfers are proven to write exactly the supplied address.

Specifications: Legacy Incentive

`incentive_specs::get_pool_info`

`incentive_specs::get_pool_count`

`incentive_specs::add_pool`

`incentive_specs::earned`

`incentive_specs::claim_reward_non_entry`

`incentive_specs::create_and_transfer_ownership`

`incentive_specs::set_admin`

`incentive_specs::set_owner`

IncentiveV2

Reward fund deposit and withdrawal prove exact balance accounting. Release rate, user effective amount, and the per-pool contribution that they compose into are proven. `claim_reward_non_entry` is proven for successful executions.

Pool management and querying. `create_funds_pool` is proven at the structural level, with its abort behavior covered and its postconditions bounded by the dynamic object creation limitation noted in Methodology. The pool freeze is proven to retain only pools whose phase is at or above the deadline, and each of the three query paths is proven to return exactly the pools matching the requested asset, option, funds address, and time window, including the vector that filters on the conjunction of asset and option.

All four of those proofs carry a loop invariant over the pool objects vector and are stated through quantifier-wrapped pure predicates, which is what keeps a filter over a growing pool vector expressible as a single postcondition.

The deprecation is proven rather than assumed. Each deposit, withdrawal, borrow, repay, and liquidation path, including the account-capability and on-behalf-of variants, is proven to abort unconditionally, so no V2 user-facing operation can complete on chain. The reward fund round trip is proven to conserve funds across a deposit and a matching withdrawal.

Specifications: IncentiveV2

<code>incentive_v2_specs::add_funds</code>
<code>incentive_v2_specs::withdraw_funds</code>
<code>incentive_v2_specs::create_funds_pool</code>
<code>incentive_v2_specs::create_and_transfer_owner</code>
<code>incentive_v2_specs::calculate_release_rate</code>
<code>incentive_v2_specs::calculate_user_effective_amount</code>
<code>incentive_v2_specs::calculate_one_from_pool</code>
<code>incentive_v2_specs::claim_reward_non_entry</code>
<code>incentive_v2_specs::freeze_incentive_pool</code>
<code>incentive_v2_specs::get_pool_from_funds_pool</code>
<code>incentive_v2_specs::get_pool_from_asset_and_option</code>
<code>incentive_v2_specs::get_active_pools</code>
<code>incentive_v2_specs::get_pool_info</code>
<code>incentive_v2_specs::get_pool_index_reward_from_incentive</code>
<code>incentive_v2_specs::version_verification</code>
<code>incentive_v2_specs::version_migrate</code>

Specifications: IncentiveV2 Deprecated Entry Points

<code>incentive_v2_specs::entry_deposit</code>
<code>incentive_v2_specs::entry_withdraw</code>
<code>incentive_v2_specs::entry_borrow</code>
<code>incentive_v2_specs::entry_repay</code>
<code>incentive_v2_specs::entry_liquidation</code>
<code>incentive_v2_specs::entry_deposit_on_behalf_of_user</code>
<code>incentive_v2_specs::entry_repay_on_behalf_of_user</code>
<code>incentive_v2_specs::deposit_with_account_cap</code>
<code>incentive_v2_specs::withdraw_with_account_cap</code>

<code>incentive_v2_specs::borrow_with_account_cap</code>
<code>incentive_v2_specs::repay_with_account_cap</code>
<code>incentive_v2_specs::withdraw</code>
<code>incentive_v2_specs::borrow</code>
<code>incentive_v2_specs::repay</code>
<code>incentive_v2_specs::liquidation</code>
<code>funds_roundtrip_scenario_specs::v2_funds_roundtrip_scenario</code>

IncentiveV3

Admin operations. Version lifecycle, market initialization, and cross-object market ID consistency checks are proven. Reward fund deposit and withdrawal prove exact balance accounting. Rule enable and disable operations and rate mutations are proven to write exactly the configured fields. Pool and reward fund creation are proven with storage market ID consistency, and the legacy incentive and reward fund constructors are proven to abort unconditionally.

User operations. The live borrow, withdrawal, repay, and liquidation paths are specified, including the account-capability and fee-charging variants, and `get_effective_balance` proves the balance the reward computation is weighted by. `parse_claimable_rewards` proves that the output vectors, covering asset coin types, reward coin types, claimable rewards, claimed rewards, and rule IDs, all have the same length as the input, using a loop invariant tracking the processing count against remaining elements.

Specifications: IncentiveV3 Administration

<code>manage_specs::create_incentive_v3</code>
<code>manage_specs::create_incentive_v3_with_storage</code>
<code>manage_specs::create_incentive_v3_pool</code>
<code>manage_specs::create_incentive_v3_reward_fund</code>
<code>manage_specs::create_incentive_v3_reward_fund_with_storage</code>
<code>manage_specs::deposit_incentive_v3_reward_fund</code>
<code>manage_specs::withdraw_incentive_v3_reward_fund</code>
<code>manage_specs::enable_incentive_v3_by_rule_id</code>
<code>manage_specs::disable_incentive_v3_by_rule_id</code>
<code>manage_specs::set_incentive_v3_reward_rate_by_rule_id</code>
<code>manage_specs::set_incentive_v3_max_reward_rate_by_rule_id</code>
<code>manage_specs::set_incentive_v3_borrow_fee_rate</code>
<code>manage_specs::incentive_v3_version_migrate</code>

<code>incentive_v3_specs::init_for_main_market</code>
<code>incentive_v3_specs::init_fund_for_market</code>
<code>incentive_v3_specs::version_verification</code>
<code>incentive_v3_specs::version_migrate</code>

Specifications: IncentiveV3 User Operations

<code>incentive_v3_specs::borrow</code>
<code>incentive_v3_specs::borrow_v2</code>
<code>incentive_v3_specs::borrow_with_account_cap</code>
<code>incentive_v3_specs::withdraw</code>
<code>incentive_v3_specs::withdraw_with_account_cap</code>
<code>incentive_v3_specs::entry_withdraw</code>
<code>incentive_v3_specs::repay_with_account_cap</code>
<code>incentive_v3_specs::liquidation</code>
<code>incentive_v3_specs::get_effective_balance</code>
<code>incentive_v3_specs::get_user_claimable_rewards</code>
<code>incentive_v3_specs::parse_claimable_rewards</code>
<code>incentive_v3_specs::update_reward_state_by_asset</code>

Reward settlement and no double claim. Reward accrual is index-based in the same way as interest, so the settlement step and the claim step must agree about how much a user is owed and must not pay it twice. We prove settlement is monotone and idempotent: the first update advances the global index, settles the user to it, and stamps the update time, and a second update at the same clock time changes nothing because the elapsed duration is zero and the user index already equals the global index.

We then prove the claim path on top of that. The first claim moves the user's claimed watermark up to the accrued total and splits exactly that difference out of the reward fund, so the fund plus the payout equals the fund at entry. A second claim at the same clock time pays out zero and leaves the fund untouched, and the global index is proven unchanged between the two.

Specifications: Reward Settlement and Claim

<code>incentive_v3_settlement_scenario_specs::update_reward_state_by_rule_settlement_scenario</code>
<code>incentive_v3_claim_scenario_specs::base_claim_reward_by_rule_double_claim_scenario</code>

E-mode

E-mode (efficiency mode) allows users to use tighter risk parameters when their collateral and debt are within a registered correlated-asset pair. We prove the full configuration surface and user-facing state transitions.

Configuration. `create_emode_asset` , exposed at both the storage and the management layer, proves all six E-mode asset fields are written exactly, and `create_emode_pair` proves the pair is registered. Each E-mode risk parameter setter (e.g. `set_emode_asset_ltv`) proves its field is written exactly and leaves the others untouched.

Market initialization. `init_emode_for_main_market` proves the E-mode table is initialized for the main market. `init_for_main_market` (in `manage`) proves the same through the management layer.

User state transitions. `enter_emode` and `exit_emode` , exposed at both the storage and the lending layer, prove the user's E-mode ID is updated. `verify_emode_support` proves the check that the target configuration supports the requested asset, and `is_passed_emode_restrictions` proves the admission predicate. The E-mode read surface is proven alongside them, so the values the health and liquidation layers consult when deciding whether the tighter parameters apply are the values that were configured.

Admission gating. E-mode is only safe if a user inside it cannot acquire a position outside the correlated pair. The gate is enforced at the ledger layer on both the collateral side and the debt side, and the two checks are not interchangeable because a pair may admit an asset as collateral without admitting it as debt. We prove each gate separately against the real storage mutators, and then prove the fence holds across a whole session: entering on an empty position, depositing the collateral leg, borrowing the debt leg, repaying, withdrawing, and exiting. The fence is asserted after every step, so it is established at entry, preserved by each gated append and each ungated removal, and vacuous again after exit.

Specifications: E-mode Admission Gating

`storage_scenario_specs::emode_gate_deposit_scenario`

`storage_scenario_specs::emode_gate_borrow_scenario`

`storage_scenario_specs::emode_session_scenario`

`toggle_scenario_specs::emode_toggle_scenario`

Specifications: E-mode

`storage_specs::create_emode_asset`

`storage_specs::create_emode_pair`

`storage_specs::set_emode_config_active`

`storage_specs::set_emode_asset_lt`

`storage_specs::set_emode_asset_ltv`

`storage_specs::set_emode_asset_liquidation_bonus`

`storage_specs::init_emode_for_main_market`

`storage_specs::enter_emode`

`storage_specs::exit_emode`

`storage_specs::verify_emode_support`

`storage_specs::is_passed_emode_restrictions`

`storage_specs::is_in_emode`

`storage_specs::get_user_emode_id`

`storage_specs::get_emode_info`

`storage_specs::get_emode_asset_info`

`manage_specs::create_emode_asset`

`manage_specs::create_emode_pair`

`manage_specs::set_emode_config_active`

`manage_specs::set_emode_asset_lt`

`manage_specs::set_emode_asset_ltv`

`manage_specs::set_emode_asset_liquidation_bonus`

`lending_specs::enter_emode`

`lending_specs::enter_emode_with_account_cap`

`lending_specs::exit_emode`

`lending_specs::exit_emode_with_account_cap`

Protocol Gating

Three independent gates stand in front of protocol state: a pause flag, a version number on each upgradeable object, and the E-mode admission fence. Their correctness has two halves. The toggle itself must behave as a plain last-write-wins switch, and the gated operations must actually abort when the gate is closed.

Toggles. We prove that setting the pause flag to a value pauses, that setting it again to the same value is a no-op, and that setting it back unpauses, on both the storage side and the oracle side. The E-mode

active flag is proven to be the same kind of toggle. Idempotency matters here because administrative operations are frequently replayed, and a toggle that accumulated state would drift.

Gates. `pause_gate_scenario_spec` proves that the gated operations abort while the protocol is paused. `version_gate_scenario_spec` proves the version guard on the lending objects, which require the stored version to be strictly below the target before a migration and to land on the current version afterwards. `oracle_version_gate_scenario_spec` proves the oracle side of the same property, where the two writers guard with a non-strict comparison and therefore admit the already-current case, which the lending-side guard rejects.

Specifications: Toggles and Gates

<code>toggle_scenario_specs::pause_toggle_scenario</code>

<code>toggle_scenario_specs::pause_gate_scenario</code>

<code>toggle_scenario_specs::version_gate_scenario</code>

<code>toggle_scenario_specs::oracle_version_gate_scenario</code>
--

<code>storage_specs::pause</code>

Version guards on every upgradeable object. Each versioned object carries its own guard and its own migration, and each is proven independently, so an upgrade cannot leave one object accepting calls at a stale version while another rejects them.

Specifications: Version Guards by Object

<code>version_specs::this_version</code>
--

<code>version_specs::next_version</code>
--

<code>version_specs::pre_check_version</code>

<code>oracle_version_specs::this_version</code>

<code>oracle_version_specs::next_version</code>

<code>oracle_version_specs::pre_check_version</code>
--

<code>flash_loan_specs::version_verification</code>

<code>flash_loan_specs::version_migrate</code>
--

<code>incentive_v2_specs::version_verification</code>

<code>incentive_v2_specs::version_migrate</code>
--

<code>incentive_v3_specs::version_verification</code>

<code>incentive_v3_specs::version_migrate</code>
--

<code>config_specs::version_verification</code>

<code>config_specs::version_migrate</code>
--

<code>oracle_specs::version_migrate</code>
<code>oracle_specs::oracle_version_migrate</code>
<code>constants_specs::version</code>

Lending Entry Points

The `lending` module exposes the primary user-facing entry points. Legacy v1 functions are proven to always abort, confirming deprecation. The current interfaces route through `IncentiveV3`.

`create_account` proves a new `AccountCap` is transferred to the transaction sender. All flash loan borrow and repay entry points are specified modulo abort, and the flash repay paths carry the cash deltas that the flash loan conservation proof composes with.

The deprecation of the v1 surface is proven rather than described. Every v1 entry point on `lending` is proven to abort unconditionally, so the only reachable route into the ledger is through `IncentiveV3`.

Specifications: Deprecated v1 Entry Points

<code>lending_specs::deposit</code>
<code>lending_specs::withdraw</code>
<code>lending_specs::borrow</code>
<code>lending_specs::repay</code>
<code>lending_specs::liquidation_call</code>
<code>lending_specs::delete_account</code>

`update_state_of_all` is proven with a loop invariant over all registered reserves using `forall!` quantifiers. The spec composes

`logic::update_state` per-asset via `include`, with uninterpreted wrappers for per-asset basic and overflow preconditions. This proves that sweeping `update_state` across every reserve completes safely when the per-asset preconditions hold universally.

Specifications: Lending Entry Points

<code>lending_specs::create_account</code>
<code>lending_specs::verify_market_storage_pool</code>
<code>lending_specs::verify_market_flash_loan_config_pool</code>
<code>lending_specs::flash_loan_with_ctx</code>
<code>lending_specs::flash_loan_with_ctx_v2</code>
<code>lending_specs::flash_loan_with_account_cap</code>
<code>lending_specs::flash_loan_with_account_cap_v2</code>

<code>lending_specs::flash_repay_with_ctx</code>
<code>lending_specs::flash_repay_with_account_cap</code>
<code>lending_specs::update_state_of_user</code>
<code>lending_specs::update_state_of_all</code>

Oracle Management

The oracle module provides price feeds consumed by health and liquidation computations. The oracle management surface is proven in full, together with the configuration store it writes to, the provider adaptors it reads from, and the runtime validation applied to every price.

Version and pause. `version_migrate` upgrades both the `OracleConfig` and `PriceOracle` objects to the current protocol version, requiring that each starts at or below the target.

Price feed lifecycle. `create_price_feed` is specified modulo abort. Every remaining price feed configuration setter is proven to write exactly the supplied value, so each configured bound the validation logic reads back is the bound an administrator wrote.

Oracle providers. Pyth, Supra, and Switchboard provider configurations each have create, enable, disable, and pair-ID-set operations proven.

Specifications: Oracle Management

<code>oracle_manage_spec::create_config</code>
<code>oracle_manage_spec::version_migrate</code>
<code>oracle_manage_spec::set_pause</code>
<code>oracle_manage_spec::create_price_feed</code>
<code>oracle_manage_spec::set_enable_to_price_feed</code>
<code>oracle_manage_spec::set_max_timestamp_diff_to_price_feed</code>
<code>oracle_manage_spec::set_price_diff_threshold1_to_price_feed</code>
<code>oracle_manage_spec::set_price_diff_threshold2_to_price_feed</code>
<code>oracle_manage_spec::set_max_duration_within_thresholds_to_price_feed</code>
<code>oracle_manage_spec::set_maximum_allowed_span_percentage_to_price_feed</code>
<code>oracle_manage_spec::set_maximum_effective_price_to_price_feed</code>
<code>oracle_manage_spec::set_minimum_effective_price_to_price_feed</code>
<code>oracle_manage_spec::set_historical_price_ttl_to_price_feed</code>
<code>oracle_manage_spec::create_pyth_oracle_provider_config</code>
<code>oracle_manage_spec::set_pyth_price_oracle_provider_pair_id</code>

oracle_manage_spec::enable_pyth_oracle_provider
oracle_manage_spec::disable_pyth_oracle_provider
oracle_manage_spec::create_supra_oracle_provider_config
oracle_manage_spec::set_supra_price_source_pair_id
oracle_manage_spec::enable_supra_oracle_provider
oracle_manage_spec::disable_supra_oracle_provider
oracle_manage_spec::create_switchboard_oracle_provider_config
oracle_manage_spec::set_switchboard_price_source_pair_id
oracle_manage_spec::enable_switchboard_oracle_provider
oracle_manage_spec::disable_switchboard_oracle_provider
oracle_manage_spec::set_primary_oracle_provider
oracle_manage_spec::set_secondary_oracle_provider

Oracle Price Feed Configuration

The `config` module manages price feed registration and lookup.

`is_price_feed_exists` is proven with a loop invariant over the feed address vector, using quantifier-wrapped pure predicates `( addr_in_table , feed_matches )` to verify the result matches an existential query over all registered feeds.

`is_secondary_oracle_available` proves the predicate that decides whether the fallback source may be consulted, and the version guard and migration for the configuration object are proven alongside them.

Each configured bound is readable both by feed identifier and directly from a feed reference, and the two forms are proven to agree. The specifications below are representative of that surface, and the full set is available in the verification dashboard.

Specifications: Oracle Price Feed Configuration (Representative)

config_specs::is_price_feed_exists
config_specs::is_secondary_oracle_available
config_specs::get_oracle_id
config_specs::get_max_timestamp_diff
config_specs::get_maximum_effective_price
config_specs::get_minimum_effective_price
config_specs::get_diff_threshold2_timer
config_specs::get_price_feed
config_specs::get_price_from_history
config_specs::get_updated_time_from_history
config_specs::is_price_feed_enable

```
config_specs::is_paused
```

Configuration ordering across a setter chain. Two orderings must hold on a price feed for the validation logic to be meaningful: the first price difference threshold must not exceed the second, and the minimum effective price must not exceed the maximum. Neither is enforced by an individual setter. We prove the orderings survive a chain of eight administrative writes on one feed, establishing both bounds and then re-setting the lower and the upper bound in turn, and we prove the two families are independent, so a threshold write never disturbs an effective price and the reverse.

Specifications: Oracle Configuration Ordering

```
oracle_scenario_specs::oracle_config_ordering_scenario
```

```
oracle_scenario_specs::oracle_price_freshness_scenario
```

Price Registry and Updates

The `oracle` module holds the registered prices that every valuation reads. Registration is proven to create a new price entry, and each update path is proven to replace the stored value and its timestamp with the supplied ones, one asset at a time or across a batch.

`get_token_price` proves the validity flag it returns is equivalent to the freshness and non-zero conditions the callers depend on, which is the fact the valuation functions consume when they assert a price is usable. The exponent a price is quoted at is proven on both the checked and the unchecked read, and `create_feeder` proves the feeder capability is minted and transferred.

Specifications: Price Registry and Updates

```
oracle_specs::register_token_price
```

```
oracle_specs::update_token_price
```

```
oracle_specs::update_token_price_batch
```

```
oracle_specs::update_price
```

```
oracle_specs::get_token_price
```

```
oracle_specs::decimal
```

```
oracle_specs::safe_decimal
```

```
oracle_specs::set_update_interval
```

```
oracle_specs::create_feeder
```

```
oracle_specs::price_object
```

```
oracle_pro_specs::get_price_from_adaptor
```

```
oracle_pro_specs::update_single_price
```

Oracle Provider Adaptors

NAVI reads prices from three external providers, and each arrives in a different encoding and at a different exponent. The adaptors normalize them to the protocol's target decimal, so a defect here would shift every valuation by a power of ten.

The scaling itself is proven separately. The exponent conversion is proven on both its plain and its guarded form, as are the two comparison primitives the validation strategy thresholds against, the unsigned difference between two sources and the relative divergence computed from it.

Specifications: Oracle Provider Adaptors

```
adaptor_pyth_specs::get_price_native
```

```
adaptor_pyth_specs::get_price_unsafe_native
```

```
adaptor_pyth_specs::get_price_to_target_decimal
```

```
adaptor_pyth_specs::get_price_unsafe_to_target_decimal
```

```
adaptor_pyth_specs::get_identifier_to_vector
```

```
adaptor_supra_specs::get_price_native
```

```
adaptor_supra_specs::get_price_to_target_decimal
```

```
adaptor_supra_specs::pair_id_to_vector
```

```
adaptor_switchboard_specs::get_price_native
```

```
adaptor_switchboard_specs::get_price_to_target_decimal
```

```
adaptor_switchboard_specs::get_aggregator_id
```

```
adaptor_switchboard_specs::get_identifier_to_vector
```

```
oracle_utils_specs::to_target_decimal_value
```

```
oracle_utils_specs::to_target_decimal_value_safe
```

```
oracle_utils_specs::abs_sub
```

```
oracle_utils_specs::calculate_amplitude
```

Oracle Price Validation Strategy

The `strategy` module validates oracle price data at query time.

`is_oracle_price_fresh` proves the freshness check: returns true if and only if the current timestamp minus the oracle timestamp is less than the maximum allowed difference. `validate_price_difference` proves the four-way classification (pass, soft-fail with timer, hard-fail, reject) based on primary-secondary price divergence against two thresholds and a duration window. `validate_price_range_and_history` proves the compound check: the price must be within effective bounds and must

not diverge from historical price beyond the allowed span percentage within the TTL window.

Specifications: Oracle Price Validation Strategy

```
strategy_specs::is_oracle_price_fresh
```

```
strategy_specs::validate_price_difference
```

```
strategy_specs::validate_price_range_and_history
```

Account Management

The `account` module stores per-user metadata in a dynamic field keyed by `AccountFieldKey`.

Field writes. `set_market_balance` proves the `AccountField` dynamic field exists after the call and the balance is updated to the supplied value. Each display setter (e.g. `set_account_name`) proves the field exists afterwards and that the named value is written exactly.

Ownership is never touched by a display write. An account capability carries both an owner and a set of user-editable display fields, and the display setters take a mutable reference to the whole capability. We prove the frame: none of the display setters changes the owner.

`account_owner` and `get_account_info` prove the reads that expose those fields, so the owner a caller reads is the owner recorded at creation.

Specifications: Account Management

```
account_specs::set_market_balance
```

```
account_specs::set_account_name
```

```
account_specs::set_account_description
```

```
account_specs::set_last_update_time
```

```
account_specs::account_owner
```

```
account_specs::get_account_info
```

Version and Constants

This matters because error codes are the protocol's externally observable failure surface, and integrators branch on them, so a renumbering would silently change what a caller believes went wrong. The specifications below are three representative examples, and the full set is available in the verification dashboard.

Specifications: Error Code Accessors (Representative)

```
error_specs::insufficient_balance
```

`error_specs::not_liquidatable`

`error_specs::exceeded_maximum_borrow_cap`

The specifications below are representative, and the full set is available in the verification dashboard. Version lifecycle functions are proven with correct abort behavior and are covered in [Protocol Gating](#).

Specifications: Protocol Constants (Representative)

`constants_specs::max_number_of_reserves`

`constants_specs::percentage_benchmark`

`constants_specs::seconds_per_year`

Arithmetic Operations

Fixed-point arithmetic underlies interest accrual, utilization calculations, and health factor computations. All operations are proven overflow-safe with exact postconditions.

Ray math. `wad_mul` and `wad_div` compute the following rounded integer formulas, with overflow preconditions:

$$\text{wad_mul}(a, b) = \lfloor (a \times b + \text{HALF_WAD}) / \text{WAD} \rfloor$$

$$\text{wad_div}(a, b) = \lfloor (a \times \text{WAD} + b/2) / b \rfloor$$

`ray_mul` and `ray_div` prove the analogous ray-precision formulas. In each case the divisor is either the fixed scale constant or an operand the precondition asserts to be nonzero, so no division-by-zero path survives. The scale constants themselves are proven, which is what allows the formulas above to be read as exact rather than as schematic.

Ray algebra. The formulas alone do not tell a reader whether the operations behave like multiplication and division. We prove the laws directly: ray multiplication is commutative, the scale constant is its identity, zero annihilates on either side, dividing by the scale constant is the identity, a zero numerator divides to zero, and a nonzero value divides itself to the scale constant. Separately, converting a wad value up to ray precision and back is proven to return the original value exactly, so the two precisions used across the protocol are interconvertible without loss.

$$\text{ray_mul}(a, b) = \text{ray_mul}(b, a)$$

$$\text{ray_mul}(a, \text{RAY}) = a$$

$$\text{ray_to_wad}(\text{wad_to_ray}(a)) = a$$

Safe math. The guarded integer operations are proven to compute exact results with overflow and division-by-zero preconditions.

Utilities. `split_coin` and `split_coin_to_balance` prove the returned coin/balance has exactly the requested value, with preconditions asserting the input is sufficient.

Specifications: Arithmetic Operations

<code>ray_math_spec::wad_mul</code>

<code>ray_math_spec::wad_div</code>

<code>ray_math_spec::ray_mul</code>

<code>ray_math_spec::ray_div</code>

<code>ray_math_spec::ray_to_wad</code>
--

<code>ray_math_spec::wad_to_ray</code>
--

<code>ray_math_spec::ray</code>

<code>ray_math_spec::wad</code>

<code>ray_math_spec::half_ray</code>

<code>ray_math_spec::half_wad</code>

<code>ray_algebra_scenario_specs::ray_algebra_scenario</code>

<code>ray_algebra_scenario_specs::wad_ray_roundtrip_scenario</code>

<code>safe_math_spec::add</code>

<code>safe_math_spec::sub</code>

<code>safe_math_spec::mul</code>

<code>safe_math_spec::div</code>

<code>safe_math_spec::mod</code>

<code>safe_math_spec::min</code>

<code>utils_spec::split_coin</code>

<code>utils_spec::split_coin_to_balance</code>
--

Interest Rate Model and Projection

Interest reaches the ledger through two layers. The `calculator` module is the kernel that accrual actually writes through, and the `dynamic_calculator` module projects the same computation forward without touching storage, so that the protocol can quote a position's current value or evaluate a hypothetical change.

Interest Rate Kernel

Utilization. Utilization drives every rate in the model, and it is a ratio, so the interesting question is the denominator.

$$U = \text{ray_div}(\text{ray_mul}(\text{tb}, \text{bi}), \text{ray_mul}(\text{ts}, \text{si}))$$

The specification requires either that the scaled borrow is zero, in which case utilization short-circuits to zero without dividing, or that the scaled supply is nonzero. There is no reachable path on which the division is by zero. The postcondition is that the result is bounded, which is what the two-slope model and the supply rate below rely on.

The two-slope borrow rate. Below the optimal utilization the rate rises on one slope, and above it a second slope is added on the excess. The model is continuous at the kink by construction.

$$\begin{aligned} U < \text{optimal}: \quad r(U) &= \text{base} + \text{ray_mul}(U, \text{multiplier}) \\ U \geq \text{optimal}: \quad r(U) &= \text{base} + \text{ray_mul}(\text{optimal}, \text{multiplier}) \\ &\quad + \text{ray_mul}(U - \text{optimal}, \text{jump}) \end{aligned}$$

The supply rate. Suppliers receive the borrow rate scaled by utilization and reduced by the reserve factor.

$$r_s = \text{ray_mul}(\text{ray_mul}(r, U), \text{RAY} - \text{reserve_factor})$$

Accrual factors. Linear interest is proven to equal `RAY` plus the rate scaled by elapsed time over a year, and compounded interest is proven to equal its third-order Taylor expansion, with each term validated by its own pure helper. Both divide only by fixed nonzero constants. Because the linear factor is a sum of non-negative terms it never falls below `RAY`, and it equals `RAY` exactly at zero elapsed time, which is the fact that makes accrual a no-op within a tick.

Valuation. `calculate_value` converts an amount to a value at the oracle price and `calculate_amount` inverts it. Both are gated on the oracle version and on the named oracle validity predicate before the price is read, and the inversion asserts the price is nonzero before dividing by it, so neither conversion has a reachable division by zero.

Specifications: Interest Rate Kernel

<code>calculator_specs::caculate_utilization</code>
<code>calculator_specs::calculate_borrow_rate</code>
<code>calculator_specs::calculate_supply_rate</code>
<code>calculator_specs::calculate_linear_interest</code>
<code>calculator_specs::calculate_compounded_interest</code>
<code>calculator_specs::calculate_value</code>
<code>calculator_specs::calculate_amount</code>

Real-Time Projection

The `dynamic_calculator` module computes real-time interest-adjusted balances, indices, utilization rates, and borrow rates without modifying storage. These functions project forward from the last-recorded state using elapsed time, enabling the protocol to evaluate hypothetical position changes.

Specifications: Real-Time Projection

<code>dynamic_calculator_specs::calculate_current_index</code>
--

<code>dynamic_calculator_specs::dynamic_user_collateral_balance</code>
--

<code>dynamic_calculator_specs::dynamic_user_loan_balance</code>
--

<code>dynamic_calculator_specs::dynamic_calculate_utilization</code>
--

<code>dynamic_calculator_specs::dynamic_calculate_borrow_rate</code>
--

<code>dynamic_calculator_specs::dynamic_calculate_supply_rate</code>
--

<code>dynamic_calculator_specs::dynamic_calculate_apy</code>
--

<code>dynamic_calculator_specs::dynamic_user_collateral_value</code>
--

<code>dynamic_calculator_specs::dynamic_user_loan_value</code>
--

<code>dynamic_calculator_specs::dynamic_user_health_collateral_value</code>

<code>dynamic_calculator_specs::dynamic_user_health_loan_value</code>

<code>dynamic_calculator_specs::dynamic_health_factor</code>
--

<code>dynamic_calculator_specs::dynamic_liquidation_threshold</code>
--

The projection matches what the ledger writes. A projection is only useful if it predicts the value the protocol will actually record.

Specifications: Projection Agrees with Accrual

<code>mi38_index_mirror_scenario_specs::mi38_accrual_formula_mirror_lemma</code>
--

<code>mi38_index_mirror_scenario_specs::mi38_prediction_is_accrual_formula_scenario</code>
--

Modeling Boundaries

Coverage is not uniform. A number of functions are modeled at the structural level and tracked in the verification dashboard.

Specifications: Bounded Coverage

<code>incentive_v2_specs::create_funds_pool</code>
--

<code>manage_specs::create_flash_loan_asset</code>
--

Conclusion and Continual Assurance

This verification effort comprises over 500 passing specifications across the NAVI lending protocol, spanning more than forty specification modules. Coverage runs from the arithmetic kernel up through reserve state and the rate model to the operations users call and the gating that stands in front of all of it. What this round adds on top of the per-function contracts is a layer of properties about whole operations. Every core operation is proven to move value conservatively between the ledger and the pool, to touch only the reserve and the user it names, and to compose into round trips that either clear exactly or lose a bounded amount that always favors the protocol. Reward settlement is proven to pay once, the projection layer is proven to predict the value the ledger will record, and the admission fence is proven to hold across an entire session rather than at a single call.

Two defects surface under that treatment, and both are reported with a closed-form witness rather than as an inconclusive result. They are stated at their exact boundary in [Counterexamples and Exact Boundaries](#). A number of functions are modeled at the structural level and tracked in the verification dashboard.

These results apply to the current deployed contract revision and specification set captured in this report. Any protocol upgrade, parameterization change that affects verified logic, or dependency-level semantic change should be treated as a trigger for re-running and refreshing the formal proofs, so verification remains a continuous assurance process rather than a one-time milestone.

This report reflects the state of the specifications and proofs as of the revision and prover version listed in the Reproducibility section. Formal verification reduces the risk of certain classes of bugs but does not guarantee the absence of all vulnerabilities; only explicitly specified properties are verified. This report is provided for informational purposes only, not as financial, legal, or investment advice. No liability is assumed for losses arising from reliance on this report. Users should perform their own due diligence before interacting with any smart contract system.

